Flight Booking Management System

(Cloud Node)

Project Report

Submitted by:

Punya Vasistha
Reg No: 2502051140

Anusha Bhalla
Reg No: 2502051148

Under the supervision of:

Dr. Krishna Kumar

Department of Computer Science and Engineering
Manipal University Jaipur
Dehmi Kalan, Near GVK Toll Plaza, Jaipur-Ajmer Expressway, Jaipur, Rajasthan 303007

Abstract

The Flight Booking Management System, branded Cloud Node, is a database-driven application that digitises the core operations of an airline reservation service, including airline and flight management, daily flight scheduling, passenger registration, seat-level ticket booking, booking details and payment recording. The objective of the project was to replace manual and error-prone reservation records with a centralised system in which every booking is tied to a registered passenger, a specific flight, a specific seat and a payment record, while staff can monitor flights, seat availability and booking trends. The methodology involved first capturing the requirements in an Entity-Relationship (ER) model, mapping it to a relational schema, and implementing the database in MySQL using SQL with primary-key, foreign-key, UNIQUE, NOT NULL, CHECK, ENUM and default-value constraints. The database consists of seven tables: Airlines, Flights, Passengers, Bookings, FlightSchedules, Payments and BookingDetails, together with two helper views (v_flights_available and v_bookings_full) and a stored procedure (sp_generate_daily_flights) that creates dated flight instances from recurring schedule templates. A lightweight Java back end (JDBC with MySQL Connector/J and the built-in HTTP server) exposes the data as a JSON API, and a single-page HTML/CSS/JavaScript front end provides a multi-step booking flow: search, passenger selection, seat map, meal choice, contact and payment, and a boarding-pass style confirmation with a PNR. Booking is executed as a single database transaction that checks flight capacity, rejects an already taken seat, inserts the booking, and records booking details and a payment, committing all of them together or none at all. The findings confirm that a normalised schema with enforced constraints and transactional booking logic is essential for keeping reservation data consistent. The project delivers a working prototype of a flight reservation system and demonstrates a practical understanding of relational database design, SQL and full-stack integration.

Contents

Chapter 1: Introduction **1**
1.1 Project Overview 1
1.2 Objectives of the Project 1
1.3 Importance and Relevance 1
1.4 Scope and Limitations 1
Chapter 2: Literature Review **3**
2.1 Airline Reservation Systems 3
2.2 Relevant Technologies 3
Chapter 3: System Design **4**
3.1 Database Design Overview 4
3.2 Entity-Relationship (ER) Diagram 4
3.3 Description of Tables and Attributes 5
3.4 Normalization 9
Chapter 4: Frontend and Backend Implementation **10**
4.1 Overview 10
4.2 Backend API 10
4.3 Landing Page and Navigation 10
4.4 Flight Search and Results 11
4.5 Booking Flow: Passenger, Seat Map and Meals 11
4.6 Contact, Payment and Confirmation 13
4.7 Bookings and Dashboards 13
Chapter 5: Results and Discussion **16**
5.1 Presentation of Results 16
5.2 Discussion of Findings 17
5.3 Comparison with Expected Outcomes 18
Chapter 6: Conclusion **19**
6.1 Summary of Project Work 19
6.2 Achievement of Project Objectives 19
6.3 Lessons Learned 19
Chapter 7: Future Work **20**
7.1 Suggestions for Future Enhancements 20
7.2 Areas for Further Research 20
References **21**

List of Figures

3.1 ER Diagram for the Flight Booking Management System....4
3.2 Relational Schema of the Flight Booking Management System....5
4.1 Landing Page and Popular Destinations....11
4.2 Flight Search and Results Page....11
4.3 Passenger Selection and Seat Map....12
4.4 Meal Selection Step....12
4.5 Contact and Payment Details....13
4.6 Booking Confirmation with PNR....13
4.7 My Bookings Page....14
4.8 Dashboards: All Flights, Open Flights, Most Popular and Partner Airlines....15

List of Tables

3.1 Airlines Table Schema....6
3.2 Flights Table Schema....6
3.3 Passengers Table Schema....6
3.4 Bookings Table Schema....7
3.5 FlightSchedules Table Schema....7
3.6 Payments Table Schema....8
3.7 BookingDetails Table Schema....8
4.1 REST-style Endpoints of FlightServer....10
5.1 Result of Query 1 (base data)....16
5.2 Result of Query 2 (base data)....17
5.3 Result of Query 3 (base data)....17
5.4 Result of Query 4 (base data)....17

Chapter 1: Introduction

1.1 Project Overview

Air travel is one of the most transaction-heavy consumer services: thousands of passengers search for flights, choose seats and pay for tickets every hour, and every one of those actions must be recorded accurately. Managing this with paper registers or disconnected spreadsheets leads to double-booked seats, duplicated passenger records, mismatched payment entries and poor visibility of flight loads. The Flight Booking Management System (Cloud Node) addresses this problem by providing a structured relational database, a Java API layer and a web interface on which a flight reservation service can run.

The system allows passengers to be registered, flights to be searched by route and date, a seat to be chosen on a live seat map, optional meal preferences and contact details to be captured, and payment to be recorded. It also gives staff dashboards for viewing all flights with seat availability, flights that have no bookings, the most frequently booked flights, and the number of flights operated by each airline.

1.2 Objectives of the Project

The primary objectives of this project were as follows:

- To design an Entity-Relationship model for a flight reservation system and convert it into a relational schema.
- To implement a normalised relational database in MySQL, using primary and foreign key constraints and CHECK, UNIQUE and ENUM constraints to ensure data integrity across airlines, flights, passengers, bookings, payments and booking details.
- To model recurring flight schedules and generate dated flight instances automatically through a stored procedure, so that any date chosen by a user has flights available.
- To implement transactional booking logic that prevents overbooking and double-booking of a seat and records the booking, its details and its payment atomically.
- To load sample data, verify the design with SQL queries, and build a Java back end and a web front end on top of the database.

1.3 Importance and Relevance

The project is relevant because it models a complete reservation workflow, from airline and flight setup to seat selection, booking and payment, which is a classic real-world example of a transaction-heavy information system. It shows how a relational database keeps interconnected entities such as flights, passengers, bookings and payments consistent, and how concurrency-sensitive operations such as seat allocation can be protected with transactions. The design can serve as a blueprint for other reservation systems such as railway, bus or event ticketing.

1.4 Scope and Limitations

The scope of the project includes airline, flight and passenger management, recurring flight schedules and generated daily flights, seat-level booking with PNR generation, cancellation of bookings, booking details (contact information, passport number, nationality, special requests), and payment recording by method (Card, UPI, NetBanking, Wallet, Cash).

Limitations of the current implementation include:

- No live payment gateway integration; the payment step is a demonstration and records a transaction reference only, with no real charge.
- There is no user login or role-based access; passengers are selected from the registered list and the dashboards are open to anyone using the application.

- Cabin class and fare variations are not modelled; every booking is charged the flight's base price.
- Seat uniqueness per flight is enforced in the application logic inside a transaction rather than by a database UNIQUE constraint, and refunds on cancellation are not processed (a cancellation only changes the booking status).
- City names are stored as text in the Flights table; a separate Airports table is not part of the current schema.
- Database credentials are read from environment variables (with a local default) and should be properly secured in a production deployment.

Chapter 2: Literature Review

This project builds upon established principles in software engineering, database management and reservation system architecture.

2.1 Airline Reservation Systems

Online travel platforms such as MakeMyTrip, Cleartrip, Skyscanner and the websites of airlines like Air India and IndiGo have demonstrated how a centralised system can replace counter-based booking. Behind them are global distribution systems and airline reservation systems that maintain inventories of flights and seats, passenger name records (PNRs) and payment information, and must remain consistent when many users try to book the same seat at the same time. The Flight Booking Management System draws inspiration from these platforms but implements the essential workflow on a smaller scale, focusing on a clean relational design and reliable booking data.

2.2 Relevant Technologies

2.2.1 MySQL

MySQL is a widely used open-source relational database management system (RDBMS) known for its reliability, speed and support for ACID transactions. Data is stored in tables of rows and columns, and relationships between tables are enforced through primary and foreign keys. This is well suited to a flight system, where a booking must refer to a valid passenger and flight, and a payment must belong to a valid booking.

2.2.2 SQL, Views and Stored Procedures

SQL (Structured Query Language) is used to create the schema and manipulate the data. MySQL provides data types such as INT, VARCHAR, DECIMAL, DATE, TIME, DATETIME and ENUM. Constraints such as PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK, AUTO_INCREMENT and DEFAULT CURRENT_TIMESTAMP are declared in the CREATE TABLE statements. Views (v_flights_available, v_bookings_full) package frequently used joins, and a stored procedure (sp_generate_daily_flights) uses a cursor and a loop to generate dated flights from schedule templates.

2.2.3 Java, JDBC and the HTTP Server

The back end is written in Java. It uses JDBC with the MySQL Connector/J driver to talk to the database and the HTTP server included in the JDK (com.sun.net.httpserver) to expose REST-style JSON endpoints and to serve the front end. Prepared statements are used for queries so that user input is passed as parameters rather than concatenated into SQL, and the booking endpoint uses manual transaction control (setAutoCommit(false), commit and rollback).

2.2.4 HTML, CSS and JavaScript

The front end, named Cloud Node, is a single HTML file with embedded CSS and JavaScript. It uses the browser's fetch API to call the Java endpoints and renders the search results, seat map, booking steps and dashboards dynamically.

Chapter 3: System Design

3.1 Database Design Overview

The heart of the Flight Booking Management System is its MySQL database, called *flight_booking*. The design is normalised to minimise redundancy and ensure logical data storage. Primary and foreign keys enforce referential integrity, so that a flight must belong to a valid airline and a booking must be linked to a valid passenger and flight. The schema consists of seven tables: Airlines, Flights, Passengers, Bookings (created by the base script) and FlightSchedules, Payments and BookingDetails (added by the extensions script), plus two views and one stored procedure.

3.2 Entity-Relationship (ER) Diagram

The ER diagram in Figure 3.1 provides a visual blueprint of the database. It illustrates the core entities, their main attributes and the relationships that connect them. Figure 3.2 shows the corresponding relational schema, with primary keys (PK) and foreign keys (FK) marked.

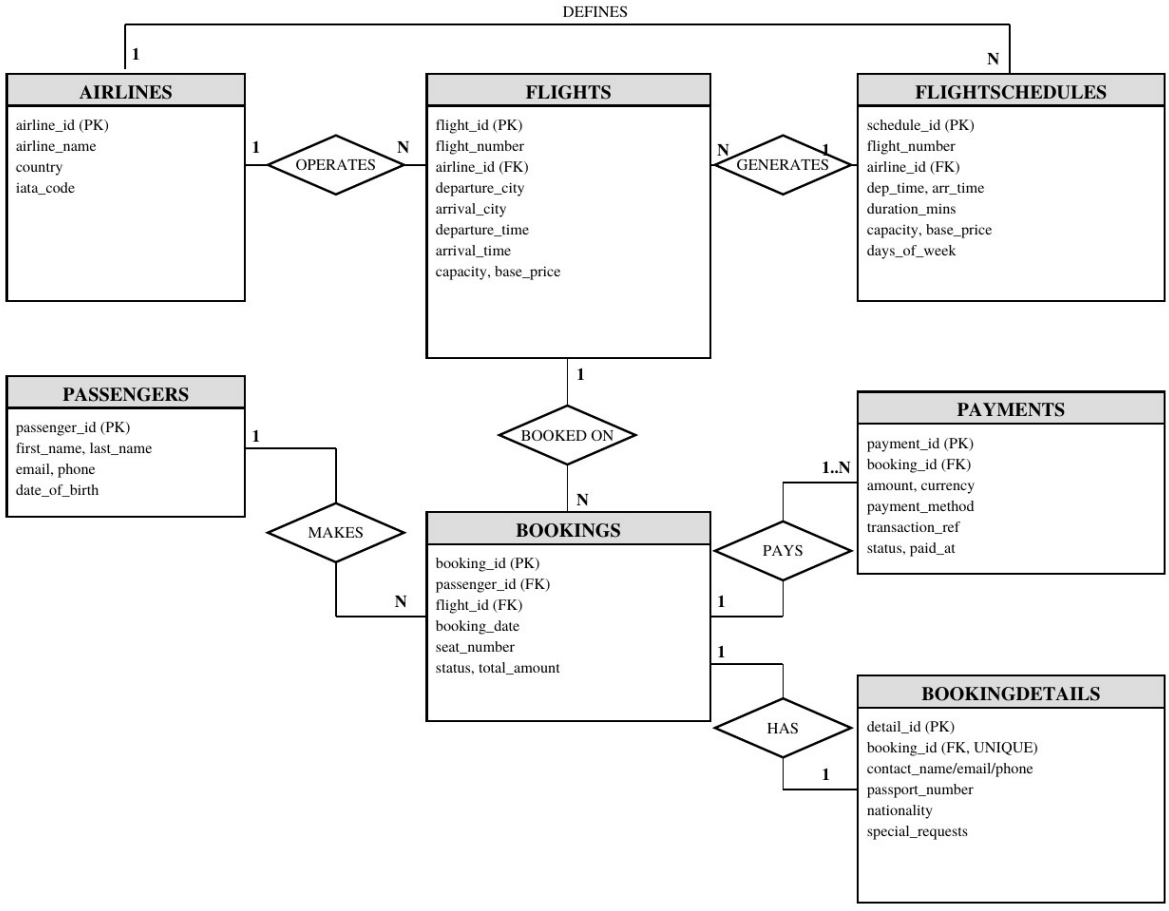


Figure 3.1: ER Diagram for the Flight Booking Management System

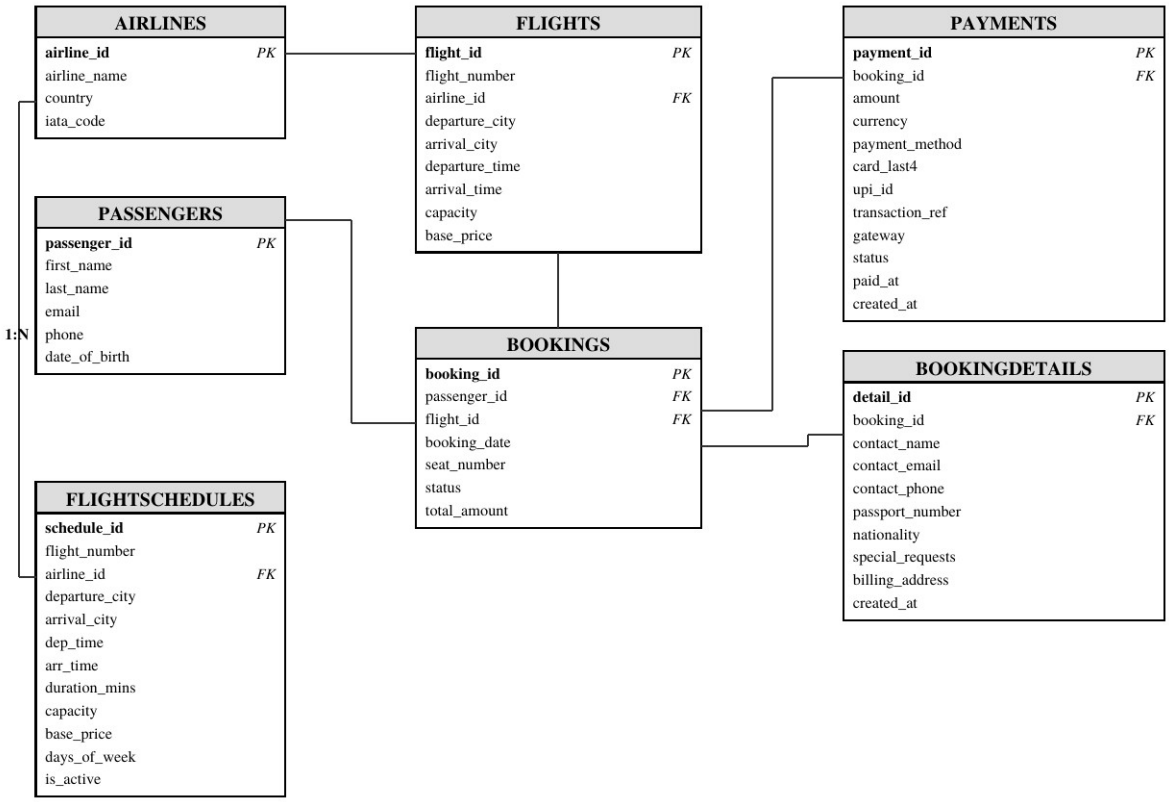


Figure 3.2: Relational Schema of the Flight Booking Management System

The key entities and relationships include:

- **Entities:** Airline, Flight, Passenger, Booking, FlightSchedule, Payment and BookingDetails.
- **One-to-Many Relationships:** An Airline operates many Flights (Operates). An Airline owns many FlightSchedules (Defines). A Passenger makes many Bookings (Makes). A Flight is booked by many Bookings (Booked On).
- **One-to-One Relationships:** In the current design each Booking has one BookingDetails row (enforced by UNIQUE on booking_id) and normally one Payment row (Pays), which is created in the same transaction as the booking.
- **Template-to-Instance Relationship:** A FlightSchedule is a recurring template (for example AI101 at 08:30 Delhi to Mumbai, every day). The stored procedure sp_generate_daily_flights creates dated Flights rows from it with the flight number suffixed by the date, such as AI101-20261025.

3.3 Description of Tables and Attributes

The database schema is composed of the following tables, which directly correspond to the CREATE TABLE statements of the MySQL scripts. NOT NULL columns are mandatory; PK marks a primary key, FK a foreign key and UQ a UNIQUE constraint.

Table 3.1: Airlines Table Schema

Field	Type	Key	Description
airline_id	INT	PK	Unique identifier for the airline (AUTO_INCREMENT).
airline_name	VARCHAR(100)	UQ	Name of the airline (NOT NULL, UNIQUE).
country	VARCHAR(50)		Country of the airline (NOT NULL).
iata_code	VARCHAR(10)	UQ	Two-character IATA code, e.g., AI, 6E (UNIQUE).

Table 3.2: Flights Table Schema

Field	Type	Key	Description
flight_id	INT	PK	Unique identifier for the flight (AUTO_INCREMENT).
flight_number	VARCHAR(20)	UQ	Flight number, e.g., AI101 or AI101-20261025 (NOT NULL, UNIQUE).
airline_id	INT	FK	References Airlines(airline_id) (NOT NULL).
departure_city	VARCHAR(50)		City of departure (NOT NULL).
arrival_city	VARCHAR(50)		City of arrival (NOT NULL).
departure_time	DATETIME		Date and time of departure (NOT NULL).
arrival_time	DATETIME		Date and time of arrival (NOT NULL).
capacity	INT		Total seats; CHECK (capacity > 0).
base_price	DECIMAL(10,2)		Base fare in INR; CHECK (base_price > 0).

Table 3.3: Passengers Table Schema

Field	Type	Key	Description
passenger_id	INT	PK	Unique identifier for the passenger (AUTO_INCREMENT).
first_name	VARCHAR(50)		First name of the passenger (NOT NULL).
last_name	VARCHAR(50)		Last name of the passenger (NOT NULL).
email	VARCHAR(100)	UQ	Email address (UNIQUE).
phone	VARCHAR(20)		Contact phone number.
date_of_birth	DATE		Date of birth of the passenger.

Table 3.4: Bookings Table Schema

Field	Type	Key	Description
booking_id	INT	PK	Unique identifier for the booking (AUTO_INCREMENT).
passenger_id	INT	FK	References Passengers(passenger_id) (NOT NULL).
flight_id	INT	FK	References Flights(flight_id) (NOT NULL).
booking_date	DATETIME		Date and time of booking (DEFAULT CURRENT_TIMESTAMP).
seat_number	VARCHAR(10)		Seat allotted, e.g., 12A.
status	ENUM		Confirmed, Cancelled, Pending or Completed (DEFAULT Confirmed).
total_amount	DECIMAL(10,2)		Amount charged for the booking.

Table 3.5: FlightSchedules Table Schema

Field	Type	Key	Description
schedule_id	INT	PK	Unique identifier for the schedule (AUTO_INCREMENT).
flight_number	VARCHAR(20)		Base flight number, e.g., AI101 (NOT NULL).
airline_id	INT	FK	References Airlines(airline_id) (NOT NULL).
departure_city, arrival_city	VARCHAR(50)		Route of the recurring flight (NOT NULL).
dep_time, arr_time	TIME		Daily departure and arrival time; arrival may fall on the next day.
duration_mins	INT		Flight duration in minutes (NOT NULL).
capacity, base_price	INT, DECIMAL		Seats and fare; both with CHECK (> 0).
days_of_week	VARCHAR(20)		Operating days, 1 = Monday to 7 = Sunday (DEFAULT '1234567').
is_active	TINYINT(1)		1 if the schedule is active (DEFAULT 1).

Table 3.6: Payments Table Schema

Field	Type	Key	Description
payment_id	INT	PK	Unique identifier for the payment (AUTO_INCREMENT).
booking_id	INT	FK	References Bookings(booking_id) (NOT NULL).
amount	DECIMAL(10,2)		Amount paid; CHECK (amount > 0).
currency	VARCHAR(3)		Currency code (DEFAULT 'INR').
payment_method	ENUM		Card, UPI, NetBanking, Wallet or Cash (NOT NULL).
card_last4	VARCHAR(4)		Last four digits of the card only.
upi_id	VARCHAR(100)		UPI handle, if paid by UPI.
transaction_ref	VARCHAR(64)	UQ	Unique transaction reference, e.g., CNPAY-20261015-0001.
gateway	VARCHAR(50)		Gateway name (DEFAULT 'CloudNodePay').
status	ENUM		Pending, Success, Failed or Refunded (DEFAULT Pending).
paid_at, created_at	DATETIME		Time of payment and of record creation (created_at DEFAULT CURRENT_TIMESTAMP).

Table 3.7: BookingDetails Table Schema

Field	Type	Key	Description
detail_id	INT	PK	Unique identifier (AUTO_INCREMENT).
booking_id	INT	FK, UQ	References Bookings(booking_id); one details row per booking.
contact_name	VARCHAR(100)		Contact person's name (NOT NULL).
contact_email	VARCHAR(100)		Contact email (NOT NULL).
contact_phone	VARCHAR(20)		Contact phone number.
passport_number	VARCHAR(30)		Passport number, for international travel.
nationality	VARCHAR(50)		Nationality of the traveller.
special_requests	VARCHAR(255)		Meal or seat requests, e.g., Vegetarian meal.
billing_address	VARCHAR(255)		Billing address.
created_at	DATETIME		Record creation time (DEFAULT CURRENT_TIMESTAMP).

Views and Stored Procedure

- **v_flights_available:** lists each flight with its airline, number of non-cancelled bookings (booked) and seats remaining (available). It is used for availability checks.
- **v_bookings_full:** a combined view of bookings with passenger, flight, airline, payment and booking details for reporting.

- **sp_generate_daily_flights(start, end):** loops over each date in the range and, for every active schedule that operates on that weekday, inserts a Flights row if it does not already exist. It was run for 25 October 2026 to 31 December 2026.

3.4 Normalization

- **First Normal Form (1NF):** All attributes are atomic; there are no repeating groups or multivalued columns. The days of operation in FlightSchedules are stored as a compact code string of weekday digits, which is a single coded value used only by the generator procedure.
- **Second Normal Form (2NF):** Every table has a single-column surrogate primary key, so no non-key attribute can depend on part of a key; there are no partial dependencies.
- **Third Normal Form (3NF):** Airline details are stored once in Airlines and referenced by airline_id from Flights and FlightSchedules. Passenger details are stored once in Passengers and referenced from Bookings. Payment data is kept separate from Bookings, and booking-time contact data is kept separate in BookingDetails, so no non-key attribute depends on another non-key attribute in the same table.
- **Design Note:** Bookings.total_amount is stored so that the price actually charged is preserved even if the flight's base price changes later, and city names are stored directly in Flights; both are deliberate simplifications that are listed under future work.

Chapter 4: Frontend and Backend Implementation

4.1 Overview

While the core of this report focuses on the database design, the application layer demonstrates the system's capabilities. The back end is a single Java class, FlightServer, that connects to MySQL through JDBC, serves the front end at http://localhost:8080 and provides the JSON API listed in Section 4.2. The front end is a single page (index.html) with the Cloud Node theme. The project is compiled and started with the run.sh script after the SQL scripts have been loaded.

4.2 Backend API

Table 4.1: REST-style Endpoints of FlightServer

Endpoint	Purpose
GET /api/flights	All flights with booked count and seats available.
GET /api/seats?flightId=N	Seat numbers already taken on a flight, for the seat map.
GET /api/passengers	List of registered passengers.
POST /api/passengers/add	Register a new passenger.
POST /api/book	Book a seat; writes Bookings and, if the extensions are installed, BookingDetails and Payments; returns booking id and PNR.
POST /api/cancel	Set a booking's status to Cancelled.
GET /api/bookings	Full list of bookings with passenger and flight details.
GET /api/unbooked-flights	Flights that have no bookings.
GET /api/top-flights	Top three flights by number of bookings.
GET /api/airline-counts	Number of flights operated by each airline.

Transactional booking logic

The /api/book endpoint disables auto-commit and performs the following steps inside one transaction: (1) read the flight's capacity and price and count its non-cancelled bookings; (2) reject the request if the flight is full; (3) reject the request if the chosen seat is already taken on that flight; (4) insert the Booking with status Confirmed; (5) generate the PNR from the booking id in the form CN000021; (6) insert a BookingDetails row when contact details are supplied; (7) insert a Payments row with a unique transaction reference; and (8) commit. If any step fails the transaction is rolled back, so a half-completed booking can never be stored.

4.3 Landing Page and Navigation

The application opens with an animated introduction and a landing page showing popular destinations. A navigation bar gives access to Book, My Bookings, Passengers, All Flights, Open Flights, Most Popular and Partner Airlines.

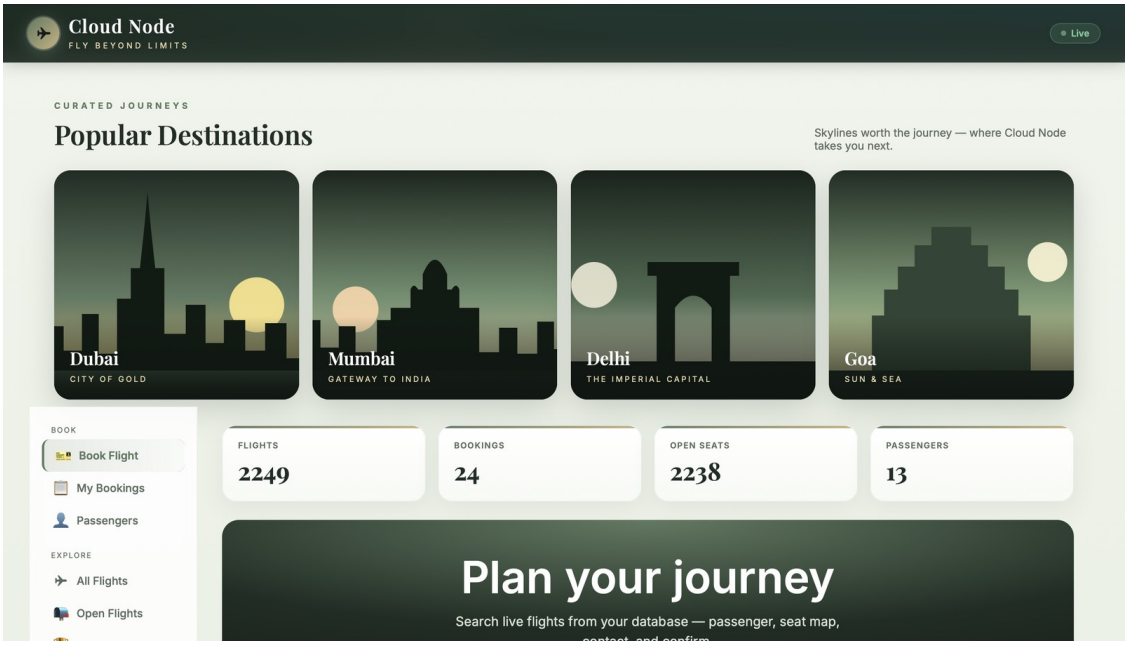


Figure 4.1: Landing Page and Popular Destinations

4.4 Flight Search and Results

Users choose the origin, destination and an optional date. Only flights that still have open seats are listed, with airline, times, duration and fare.

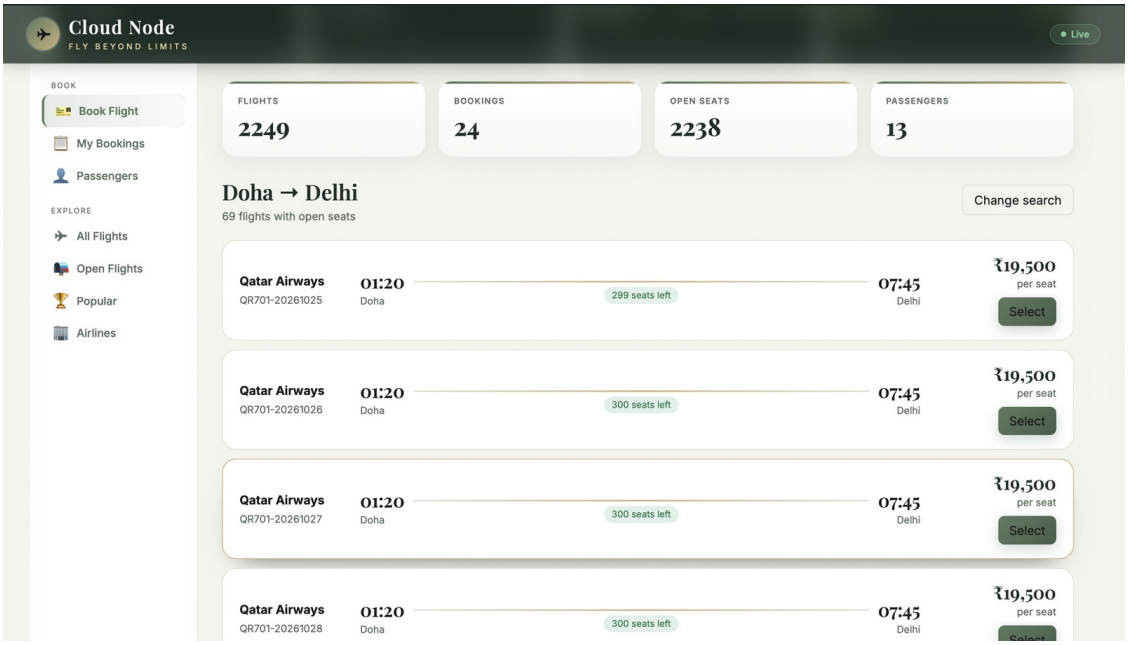


Figure 4.2: Flight Search and Results Page

4.5 Booking Flow: Passenger, Seat Map and Meals

After a flight is selected the user chooses a registered passenger (or adds a new one under Passengers), picks a seat on the cabin grid where taken seats are disabled and extra-legroom rows are highlighted, and optionally selects a complimentary meal, which is saved as a special request.

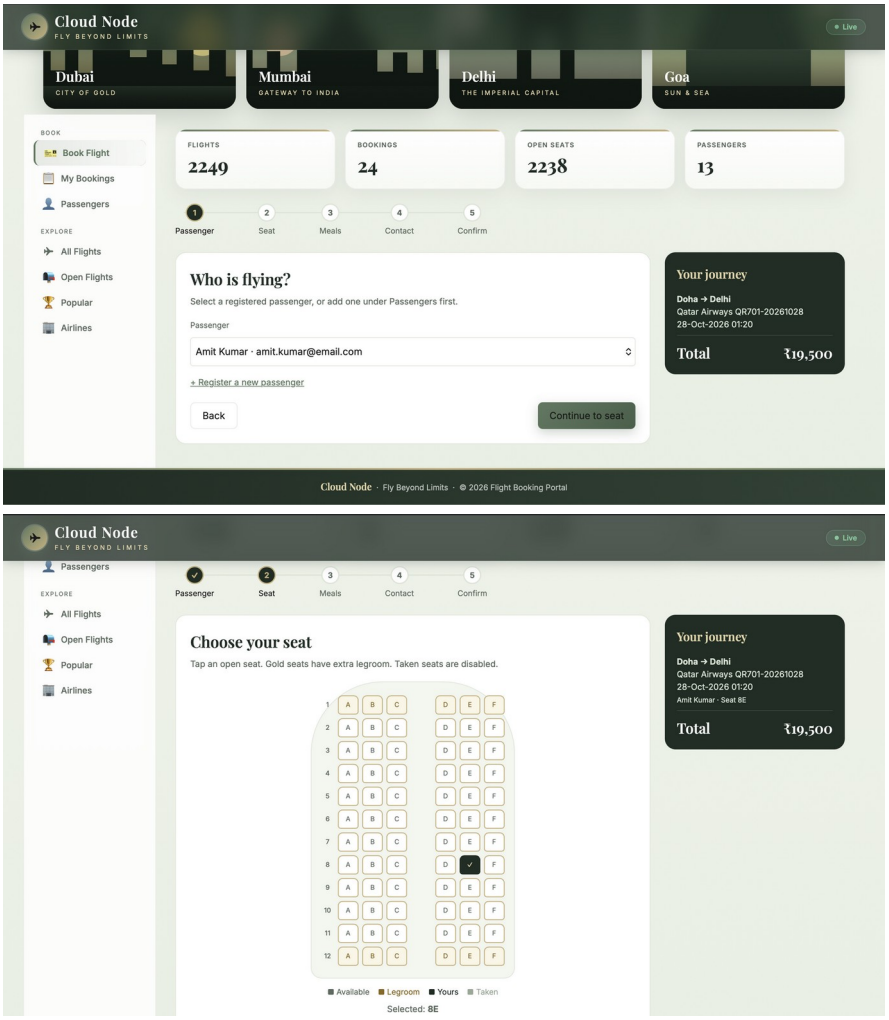


Figure 4.3: Passenger Selection and Seat Map

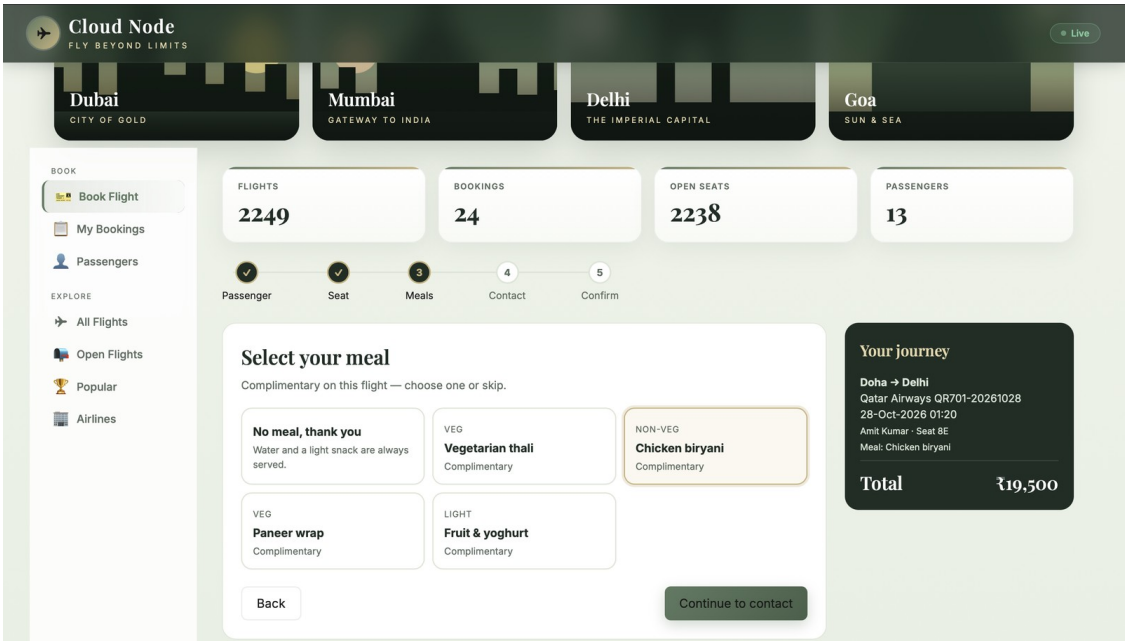


Figure 4.4: Meal Selection Step

4.6 Contact, Payment and Confirmation

The user enters contact details and chooses a payment method (UPI, Card, NetBanking or Wallet). On confirmation the booking is written to the database and a boarding-pass style card with the PNR is shown.

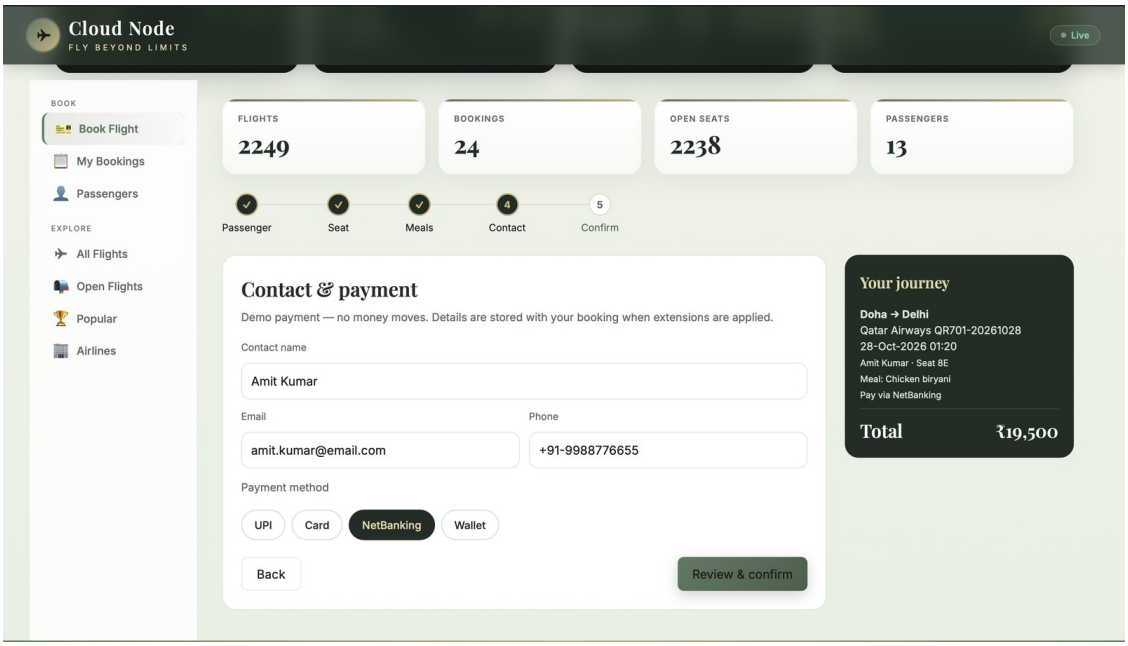


Figure 4.5: Contact and Payment Details

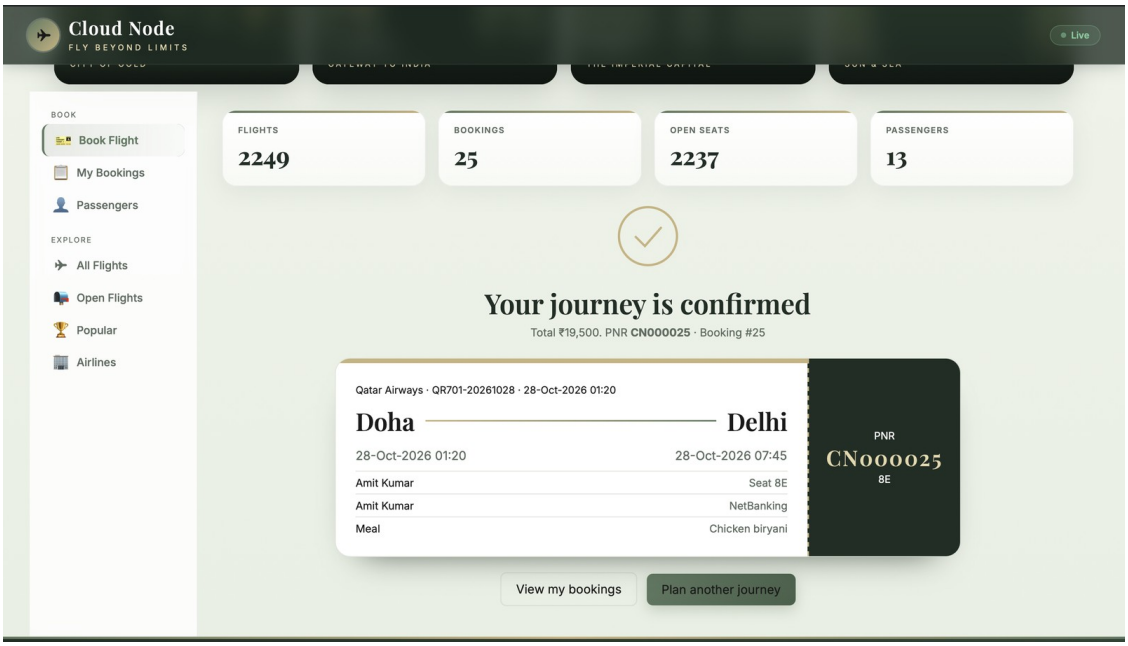


Figure 4.6: Booking Confirmation with PNR

4.7 Bookings and Dashboards

The My Bookings page lists reservations with their status and allows a booking to be cancelled. The remaining pages show live data from the database: all flights with seat availability, flights with no bookings, the three most booked flights, and the number of flights per airline.

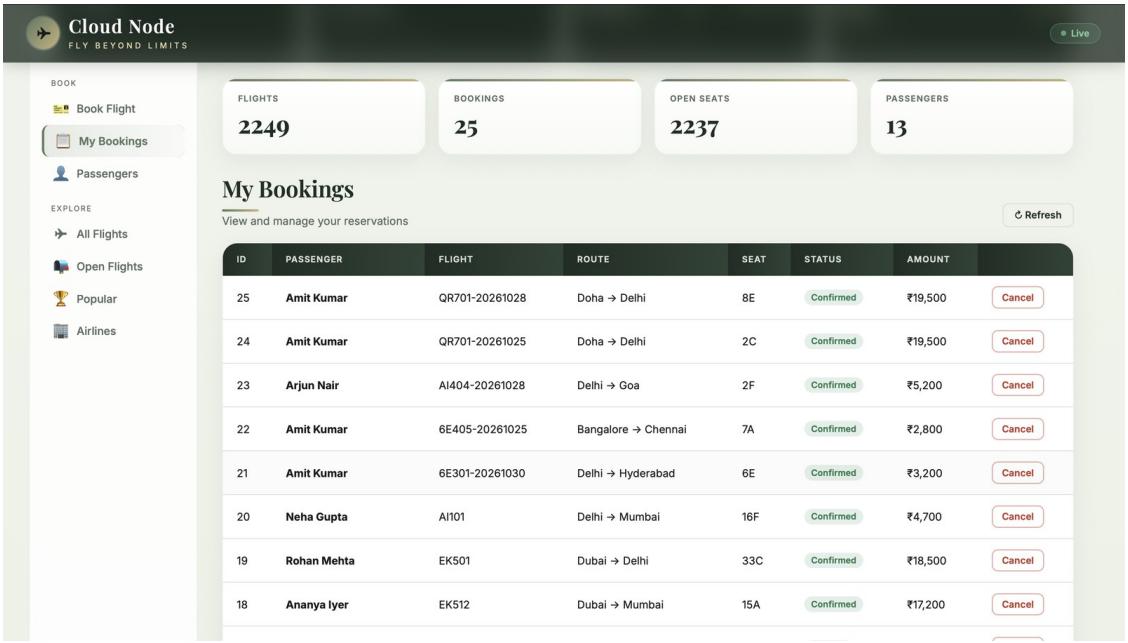


Figure 4.7: My Bookings Page

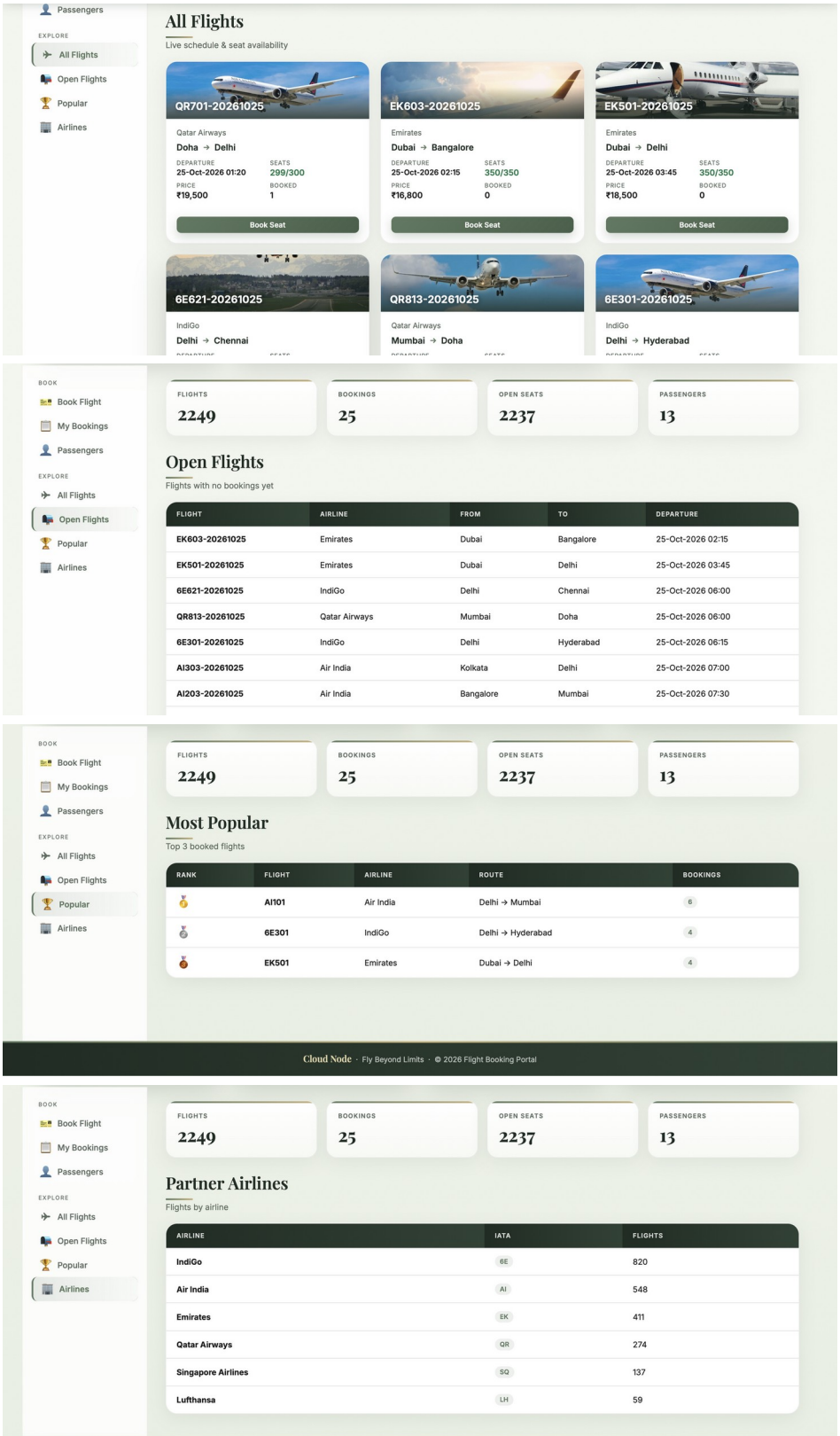


Figure 4.8: Dashboards: All Flights, Open Flights, Most Popular and Partner Airlines

Chapter 5: Results and Discussion

5.1 Presentation of Results

The project resulted in a functional flight booking database and application. The system correctly implements the logic defined by the schema.

- **Data Integrity:** Primary and foreign key constraints reject invalid data. It is impossible to create a Booking for a non-existent passenger or flight, a Payment or BookingDetails row for a non-existent booking, or a flight with zero capacity or a non-positive price (CHECK constraints). Duplicate airline names, flight numbers, passenger emails and transaction references are rejected by UNIQUE constraints.
- **Sample Data:** The base script loads 6 airlines, 15 flights, 12 passengers and 20 bookings. The extensions script adds 34 flight schedule templates, 20 payments and 20 booking details rows. Running the stored procedure for 25 October to 31 December 2026 generates 2,234 dated flights, so any date picked in the front end has flights.
- **Transaction Safety:** A booking that fails midway (for example a seat taken by someone else) leaves no partial rows, because the whole booking is rolled back.
- **Query Verification:** The required test queries and additional analytical queries were run and checked against the sample data, as shown below.

Query 1: Number of flights for each airline

```
SELECT a.airline_name, a.iata_code, COUNT(f.flight_id) AS total_flights
FROM Airlines a LEFT JOIN Flights f ON a.airline_id = f.airline_id
GROUP BY a.airline_id, a.airline_name, a.iata_code
ORDER BY total_flights DESC;
```

Table 5.1: Result of Query 1 (base data)

Airline	IATA	Total flights
Air India	AI	4
IndiGo	6E	4
Emirates	EK	3
Qatar Airways	QR	2
Singapore Airlines	SQ	1
Lufthansa	LH	1

Query 2: Flights that have no bookings

```
SELECT f.flight_number, a.airline_name, f.departure_city, f.arrival_city
FROM Flights f JOIN Airlines a ON f.airline_id = a.airline_id
LEFT JOIN Bookings b ON f.flight_id = b.flight_id
WHERE b.booking_id IS NULL ORDER BY f.departure_time;
```

Table 5.2: Result of Query 2 (base data)

Flight	Airline	From	To
6E405	IndiGo	Bangalore	Chennai
LH901	Lufthansa	Frankfurt	Mumbai
AI303	Air India	Kolkata	Delhi
6E512	IndiGo	Hyderabad	Mumbai
AI404	Air India	Delhi	Goa
6E620	IndiGo	Chennai	Delhi
EK603	Emirates	Dubai	Bangalore
QR812	Qatar Airways	Doha	Mumbai

Query 3: Top 3 most frequently booked flights

```
SELECT f.flight_number, a.airline_name,
       CONCAT(f.departure_city,' -> ',f.arrival_city) AS route,
       COUNT(b.booking_id) AS number_of_bookings
FROM Flights f JOIN Airlines a ON f.airline_id = a.airline_id
JOIN Bookings b ON f.flight_id = b.flight_id
GROUP BY f.flight_id, f.flight_number, a.airline_name,
         f.departure_city, f.arrival_city
ORDER BY number_of_bookings DESC LIMIT 3;
```

Table 5.3: Result of Query 3 (base data)

Flight	Airline	Route	Bookings
AI101	Air India	Delhi → Mumbai	6
EK501	Emirates	Dubai → Delhi	4
6E301	IndiGo	Delhi → Hyderabad	4

Query 4: Bookings by status

```
SELECT status, COUNT(*) AS count, ROUND(SUM(total_amount),2) AS total_amount
FROM Bookings GROUP BY status ORDER BY count DESC;
```

Table 5.4: Result of Query 4 (base data)

Status	Count	Total amount (INR)
Confirmed	19	194,300.00
Pending	1	4,500.00

Other queries in the script include passengers who booked more than one flight (all eight passengers with repeat bookings appear, each with two bookings), seat availability for flights departing from Delhi, revenue per flight, passengers flying between two dates, and the most popular routes.

5.2 Discussion of Findings

The development process highlighted the importance of initial database design. The ER model guided the schema, and separating payments and booking details from the Bookings table kept each table focused and allowed the extensions to be added without modifying existing tables or rows. Modelling recurring schedules as templates and generating dated flights with a stored procedure removed the need

to insert thousands of flight rows by hand. The application checks for the optional tables (Payments, BookingDetails) before writing to them, so the system also works when only the base script has been loaded.

5.3 Comparison with Expected Outcomes

The project met the initial objectives. The final schema covers airlines, flights, schedules, passengers, bookings, payments and booking details, and the application provides an end-to-end booking journey with a database-backed seat map. Some refinements remain, such as cabin classes, user authentication and a database-level unique constraint on flight and seat, which are listed under future work.

Chapter 6: Conclusion

6.1 Summary of Project Work

This project covered the design and implementation of a flight reservation database and application: designing the ER diagram and relational schema, implementing it in MySQL with constraints, views and a stored procedure, loading sample data and verifying it with queries, building a Java JDBC back end with a transactional booking endpoint, and developing a multi-step web front end.

6.2 Achievement of Project Objectives

All primary objectives were achieved. The system correctly captures the interactions between airlines, flights, passengers, bookings, payments and booking details, prevents overbooking and double-booking of seats within a transaction, and gives users a complete flow from search to a confirmed PNR.

6.3 Lessons Learned

The key lesson is that in a transaction-heavy application the database is the core of the system's logic and integrity, not just storage. Wrapping a multi-table write in a single transaction is what keeps bookings, details and payments consistent. Keeping the ER diagram, the SQL scripts, the API and the front end consistent with each other is equally important, since differences between them cause confusion later in development.

Chapter 7: Future Work

7.1 Suggestions for Future Enhancements

- **Stronger Constraints:** Add a UNIQUE constraint on (flight_id, seat_number) for active bookings, CHECK constraints on dates of birth and flight times (arrival after departure), and indexes on frequently searched columns such as departure_city, arrival_city and departure_time.
- **Cabin Classes and Fares:** Add a Class table and dynamic pricing so that economy, premium and business fares differ.
- **Airports Table:** Replace city text with an Airports table (code, city, country) referenced by Flights and FlightSchedules.
- **Authentication and Roles:** Add user accounts with salted password hashes, and separate passenger and administrator roles.
- **Cancellation and Refunds:** Link cancellations to refund records and update the payment status to Refunded.
- **Payment Gateway Integration:** Integrate a service such as Razorpay or Stripe for real, secure payments.
- **Deployment:** Move to Spring Boot with a connection pool and environment-based configuration for production use.

7.2 Areas for Further Research

Future research could explore database performance optimisation at scale, including indexing strategies, partitioning of the Flights table by date and replication to handle peak booking loads. Another area is the use of machine learning for demand forecasting and dynamic fare optimisation, or for predicting delays and improving schedule punctuality.

References

[1] Oracle Corporation. (n.d.). MySQL Reference Manual. Retrieved from https://dev.mysql.com/doc/

[2] Elmasri, R., & Navathe, S. B. (2016). Fundamentals of Database Systems (7th ed.). Pearson.

[3] Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database System Concepts (7th ed.). McGraw-Hill.

[4] Oracle Corporation. (n.d.). MySQL Connector/J Developer Guide. Retrieved from https://dev.mysql.com/doc/connector-j/en/

[5] Oracle Corporation. (n.d.). Java Platform, Standard Edition Documentation: JDBC and com.sun.net.httpserver. Retrieved from https://docs.oracle.com/en/java/

[6] Mozilla. (n.d.). MDN Web Docs: Fetch API. Retrieved from https://developer.mozilla.org/

[7] International Air Transport Association. (n.d.). Airline and Airport Code Search (IATA codes). Retrieved from https://www.iata.org/